Лабораторная работа №2

по дисциплине «Программирование на Си»

Обработка одномерных статических массивов

Кострицкий А. С., Ломовской И. В.

Москва — 2026 — TS2603081824

Содержание

1 Цель работы **1**

2 Общее задание **2**
2.1 Задача №1 2
2.2 Задача №2 2
2.3 Задача №3 3
2.4 Задача №4 3
2.5 Задача №5 4
2.6 Примечания 5

3 Взаимодействие с системой тестирования **5**

1 Цель работы

Целью лабораторной работы является знакомство студентов со статическими одномерными массивами, адресной арифметикой и классическими алгоритмами программирования, такими как поиск минимума и максимума, накопление суммы и произведения, вставка и удаление элементов массива, etc.

2 Общее задание

При решении каждой задачи реализуйте программу, которая принимает у пользователя элементы целочисленного статического массива и выполняет его обработку в соответствии с вариантом. Максимальное количество элементов, которое может ввести пользователь, равно десяти.

2.1 Задача №1

Варианты

0. Найти и вывести на экран сумму чётных элементов массива.
1. Найти и вывести на экран произведение нечётных элементов массива.
2. Найти и вывести на экран среднее арифметическое отрицательных элементов массива.
3. Найти и вывести на экран среднее геометрическое положительных элементов массива.

2.2 Задача №2

Варианты

0. Сформировать и вывести на экран новый массив, в который сначала скопировать элементы исходного массива, большие среднего арифметического.
1. Сформировать и вывести на экран новый массив, в который сначала скопировать элементы исходного массива, являющиеся простыми числами.
2. Сформировать и вывести на экран новый массив, в который сначала скопировать элементы исходного массива, которые начинаются и заканчиваются на одну и ту же цифру.
3. Сформировать и вывести на экран новый массив, в который сначала скопировать элементы исходного массива, являющиеся числами Армстронга.

2.3 Задача №3

Варианты

0. Удалить из исходного массива все элементы, которые являются числами-палиндромами. Ситуацию, когда массив после операции не изменился, ошибочной не считать.

1. Вставить в исходный массив после каждого элемента, кратного трём, очередное число Фибоначчи. Рекурсивных функций не использовать, положить $\mathrm{Fib}_0 = 0$, $\mathrm{Fib}_1 = 1$. Ситуацию, когда массив после операции не изменился, ошибочной не считать.

2. Удалить из исходного массива все элементы, которые являются полными квадратами. Ситуацию, когда массив после операции не изменился, ошибочной не считать.

3. Вставить в исходный массив после каждого положительного элемента реверс этого же элемента. Ситуацию, когда массив после операции не изменился, ошибочной не считать.

2.4 Задача №4

Необходимо организовать ввод массива *по концевому признаку*. В качестве концевого признака выступает любая ошибка, которая возникает при очередном вводе значения с помощью функции `scanf`. Например, при вводе последовательности

```
1 2 3 a
```

должен быть сформирован массив из трёх элементов со значениями

$$A_0 = 1,\ A_1 = 2,\ A_2 = 3.$$

В случае, если массив уже заполнен, а концевой признак ещё не наступил, ввод элементов в массив прекращается. Такое «переполнение» не считать ошибкой: программа должна обработать полученный массив из десяти элементов, но функция `main` в такой ситуации должна вернуть специальный код возврата, равный 100.

Варианты

0. Вывести на экран массив, упорядочив по возрастанию с помощью сортировки *пузырьком* (англ. *Bubble Sort*).

1. Вывести на экран массив, упорядочив по возрастанию с помощью сортировки *вставками* (англ. *Insertion Sort*).

2. Вывести на экран массив, упорядочив по возрастанию с помощью сортировки *выбором* (англ. *Selection Sort*).

2.5 Задача №5

При решении пятой задачи в методических целях запрещено использовать выражения вида «`a[i]`» и вообще квадратные скобки. Вместо указанного выражения используется выражение «`*pa`», где `pa` — указатель на элемент массива с индексом i (именно на i-ый элемент, а не выражение вида «`*(pa + i)`»). Также нельзя передавать как аргумент размер массива в элементах.

Вместо этого предлагается использовать пару указателей: на первый (нулевой) элемент массива и на элемент массива, расположенный за последним. Массив, для которого эти два указателя совпадают, должен интерпретироваться как пустой массив.

Варианты

0. Вычислить и вывести на экран значение

$$\max\left(A_0 + A_{n-1},\ A_1 + A_{n-2},\ A_2 + A_{n-3},\ \ldots,\ A_{\frac{n-1}{2}} + A_{\frac{n}{2}}\right),$$

   где n — размер массива.

1. Вычислить и вывести на экран значение

$$X_0 \cdot Y_0 + X_1 \cdot Y_1 + \cdots + X_{k-1} \cdot Y_{k-1},$$

   где n — размер массива, X — отрицательные элементы массива в порядке следования, Y — положительные элементы массива в обратном порядке, p — количество положительных элементов, q — количество отрицательных элементов, $k = \min(p,\ q)$.

2. Вычислить и вывести на экран значение

$$A_0 + A_0 \cdot A_1 + A_0 \cdot A_1 \cdot A_2 + \cdots + (A_0 \cdot A_1 \cdot A_2 \cdot \cdots \cdot A_m),$$

   где n — размер массива, m — либо индекс первого отрицательного элемента, либо значение $n - 1$, если в массиве отрицательных элементов нет.

3. Вычислить и вывести на экран значение

$$\min\left(A_0 \cdot A_1,\ A_1 \cdot A_2,\ A_2 \cdot A_3,\ \ldots,\ A_{n-3} \cdot A_{n-2},\ A_{n-2} \cdot A_{n-1}\right),$$

где n — размер массива.

4. Найти и вывести на экран количество уникальных чисел в массиве. Пример: в массиве (1, 2, 3, 4) четыре уникальных числа, в массиве (1, 3, 3, 4, 1, 5, 2, 2, 1) — пять уникальных чисел.

2.6 Примечания

1. *Статические массивы* следует отличать от *массивов переменной длины* (англ. *Variable Length Array, VLA*). Во избежание случайного использования последних при компиляции программы необходимо указывать ключ `Wvla`.

2. Для реализации каждой из задач этой лабораторной работы необходимо выделить несколько осмысленных функций. Необходимо предусмотреть обработку ошибочных ситуаций.

3. Под *вводом массива* подразумевается, если не указано иное, ввод количества вводимых элементов и самих элементов по порядку. Ввод неверного количества элементов следует считать исключительной ситуацией.

4. Под *выводом массива* подразумевается вывод его элементов без указания их общего количества.

5. Ситуации, когда решение задачи не может быть получено, следует считать исключительными. Например, если нужно подсчитать количество чётных элементов массива, а таких элементов в массиве нет.

   Помните, что в случае возникновения ошибочной ситуации программа должна не только выдавать соответствующее сообщение, но и возвращать ненулевой код возврата.

3 Взаимодействие с системой тестирования

1. Исходный код лабораторной работы размещается студентом в ветви `lab_LL`, а решение каждой из задач — в отдельной папке с названием вида `lab_LL_PP_CC`, где LL — номер лабораторной, PP

— номер задачи, CC — вариант студента. Если дана общая задача без вариантов, решение следует сохранять в папке с названием вида `lab_LL_PP`.

Пример: решения восьми задач седьмого варианта пятой лабораторной размещаются в папках `lab_05_01_07`, `lab_05_02_07`, `lab_05_03_07`, ..., `lab_05_08_07`.

2. Исходный код должен соответствовать оглашённым в начале семестра правилам оформления.

3. Если для решения задачи студентом создаётся отдельный проект в IDE, разрешается поместить под версионный контроль файлы проекта, добавив перед этим необходимые маски в список игнорирования. Старайтесь добавлять маски общего вида. Для каждого проекта должны быть созданы, как минимум, два варианта сборки: `Debug` — с отладочной информацией, и `Release` — без отладочной информации.

4. Сборка проекта на сервере происходит с помощью компилятора gcc с ключами `std=c99`, `Wall`, `Werror`, `Wpedantic`, `Wextra`.

   При сборке проектов, в которых используются типы с плавающей точкой, дополнительно указываются флаги `Wfloat-equal` и `Wfloat-conversion`.

   При сборке проектов лабораторных работ, в которых запрещено использовать массивы переменной длины (VLA), дополнительно указывается флаг `Wvla`.

   Если в Вашей программе используются математические функции из стандартной библиотеки, в Linux команда компиляции Вашей программы должна включать ключ `lm`, указывающий компилятору на явную компоновку математической библиотеки, которая в Linux не добавляется по умолчанию, в отличие от оставшейся части стандартной библиотеки.

   Пример:

```
gcc -std=c99 \
-Wall -Werror -Wpedantic -Wextra \
-o app.exe \
main.c \
-lm
```

5. Крайне рекомендуется с некоторой периодичностью дополнительно собирать проект с помощью компилятора clang с тем же набором флагов.

6. Крайне рекомендуется проводить анализ проекта с помощью одного или нескольких статических анализаторов, которые рассматриваются в рамках практикума. Помните, что рекомендации статанализатора нужно принимать или отвергать обоснованно.

7. Для каждой программы ещё до реализации студентом подготавливаются и помещаются под версионный контроль в подпапку `func_tests/data/` функциональные тесты, демонстрирующие её работоспособность.

   Позитивные входные данные следует располагать в файлах вида `pos_TT_in.txt`, выходные — в файлах вида `pos_TT_out.txt`, аргументы командной строки при наличии — в файлах вида `pos_TT_args.txt`, где TT — номер тестового случая.

   Негативные входные данные следует располагать в файлах вида `neg_TT_in.txt`, выходные — в файлах вида `neg_TT_out.txt`, аргументы командной строки при наличии — в файлах вида `neg_TT_args.txt`, где TT — номер тестового случая.

   Разрешается помещать под версионный контроль в подпапку `func_tests/scripts/` сценарии автоматического прогона функциональных тестов. Если Вы используете при автоматическом прогоне функциональных тестов сравнение строк, не забудьте проверить используемые кодировки. Помните, что UTF-8 и UTF-8(BOM) — две разные кодировки.

   Под версионный контроль в подпапку `func_tests/` также помещается файл `readme.md` с описанием в свободной форме содержимого каждого из тестов. Вёрстка файла на языке Markdown обязательной не является, достаточно обычного текста.

   Пример: восемь позитивных и шесть негативных функциональных тестов без дополнительных ключей командной строки должны размещаться в файлах `pos_01_in.txt`, `pos_01_out.txt`, ..., `neg_06_out.txt`. В файле `readme.md` при этом может содержаться следующая информация:

```
# Тесты для лабораторной работы №LL

## Входные данные
Целые a, b, c

## Выходные данные
Целые d, e

## Позитивные тесты:
- 01 - треугольники существуют;
- 02 - в качестве первого числа нуль;
...
- 08 - все три числа равны.

## Негативные тесты:
- 01 - вместо первого числа идёт буква;
- 02 - вместо второго числа идёт буква;
...
- 06 - вводятся слишком большие числа.
```

8. Если не указано обратное, успешность ввода должна контролироваться. При первом неверном вводе программа должна прекращать работу с ненулевым кодом возврата.

9. Вывод программы может содержать текстовые сообщения и числа. Если не указано иное, тестовая система анализирует числа в потоке вывода, поэтому они могут быть использованы только для вывода результатов — использовать числа в информационных сообщениях запрещено.

   Пример: сообщение «Input point 1:» будет неверно воспринято тестовой системой, а сообщения «Input point A:» или «Input first point:» — правильно.

   Тестовая система вычленяет из потока вывода числа, обособленные пробельными символами.

   Пример: сообщения «a=1.043» и «a = 1.043.» будут неверно восприняты тестовой системой, а сообщения «a: 1.043» или «a = 1.043» — правильно.

10. Если не указано обратное, числа двойной точности следует выводить, округляя до шестого знака после точки.